

Intro to ML Systems & Data Engineering

Overview

- **explain the gap between machine learning (ML) models and products**
- **contrast engineering and data over just algorithms**
- **identify how to leverage cloud technologies for hands-on practice**
- **recognize the specifications and requirement on real-world enterprise ML scenarios**

Course Requirements

- **python 3.9+ running in a virtual environment**
- **git + github**
- **VS Code, PyCharm, or equivalent IDE**
- **jupyter notebook**
- **azure account**

Readings

Sato, D. (n.d.). *Continuous delivery for machine learning*. martinowler.com.

<https://martinfowler.com/articles/cd4ml.html>

Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J., & Dennison, D. (2015). Hidden technical debt in machine learning systems.

https://papers.nips.cc/paper_files/paper/2015/file/86df7dcfd896fcdf2674f757a2463eba-Paper.pdf

Async	
Intro to ML Systems and Data Engineering	
Prototype vs Production	• prototype ○ answers “can this approach work?” • production ○ must continuously ▪ process data ▪ save predictions ▪ handle failures ▪ scale with demand ▪ protect secrets ▪ remain maintainable
ML Lifecycle	these stages form a cycle rather than a one-time workflow • problem farming ○ define the prediction or decision problem, business objective, success metrics, and operational constraint • data collection and preparation ○ acquire, clean, transform, validate, and continuously process data • model development ○ select algorithms, train models, tune parameters, and evaluate performance • deployment

	○ make predictions available through APIs, applications, cloud services, or other systems ● monitoring and maintenance ○ track model and system performance, detect changes in data, respond to failures, and retrain when necessary ○ monitoring may reveal new problems that require revisiting the data, model, or even the original problem definition
ML Systems vs. Traditional Software	● machine learning introduces challenges that are less prominent in traditional software development ○ model outputs are often probabilistic rather than deterministic ○ system behavior can change when the underlying data changes ○ production systems must manage multiple evolving artifacts—including: ▪ code ▪ datasets ▪ features ▪ trained models ○ testing therefore includes not only software correctness, but also: ▪ statistical model performance ▪ data validation ▪ latency ▪ throughput ▪ bias ▪ other operational measures

Core Components of a Production ML System	• a typical ML system contains several interconnected components: ○ data ingestion pipelines ▪ batch ▪ streaming ▪ on-demand data ○ feature engineering and storage for transforming raw data into usable model inputs ○ training pipelines that make model training repeatable and reproducible ○ model serving infrastructure ▪ APIs ▪ containers ▪ managed cloud endpoints ○ monitoring and feedback loops ▪ detect problems ▪ collect new information for future training • Together, these components turn a trained model into a continuously operating system.
Why Data Engineering Matters	• production ML depends heavily on the quality and availability of data • poor, incomplete, biased, or corrupted data can undermine even an excellent model • data engineering therefore focuses on: ○ reliable ingestion ○ validation

	○ storage ○ transformation ○ scheduling ○ integration across potentially very large datasets ● important data-related risks include: ○ distribution shift ○ data poisoning ○ poor label quality ○ privacy / compliance constraints ● designing ML systems requires anticipating these failure modes instead of treating data as a fixed input
The Role of Cloud Infrastructure	● cloud platforms make many production ML workloads easier to operate by providing: ○ elastic computing resources ○ managed services ○ geographic distribution ○ autoscaling ○ integrated data ecosystems ● these capabilities allow teams accomplish within a common infrastructure: ○ training large models ○ serving predictions at scale ○ connect: ▪ storage ▪ analytics ▪ ML workflows

Key Takeaway	• production machine learning is a systems engineering problem, not just a modeling problem • model accuracy remains important, but a useful ML product must also be: ○ reliable ○ scalable ○ maintainable ○ secure ○ supported by high-quality data
ML Success Hinges on Hidden Infrastructure	
Engineering Mindset and Real-World ML Systems	• useful ML system must be designed not only around model quality, but also around: ○ reliability ○ security ○ maintainability ○ data quality ○ infrastructure ○ integration with the surrounding application • in practice: ○ the model itself may represent only a small portion of the overall system • much of the work lies in: ○ supporting pipelines ○ features

	○ testing ○ infrastructure
An Engineering Mindset	• this course encourages thinking beyond experimentation and asking questions such as: ○ will this work reliably in production? ○ how will it be maintained? ○ is it secure? ○ what happens when the data or operating environment changes? ○ how will users or other systems interact with the model? • the goal is to connect ML theory with practical implementation using modern cloud systems rather than treating deployment as an afterthought
Real-World Systems Have Different Requirements	• system requirements depend on the application: ○ recommendation systems combine: ▪ behavioral data pipelines ▪ low-latency serving ▪ feedback loops that adapt as user preferences change ○ fraud detection requires: ▪ real-time streaming data ▪ rapidly computed features ▪ very low prediction latency ▪ extremely high availability

- predictive maintenance often relies on:
 - large-scale sensor data
 - batch processing
 - integration with operational workflows such as maintenance scheduling
- computer vision systems may require:
 - large-scale storage
 - GPU inference
 - high throughput
 - in sensitive domains such as medicine—strong requirements for:
 - accuracy
 - explainability
 - security
 - compliance
- there is no single definition of a “good” ML system
- different applications prioritize different qualities
 - latency
 - scale
 - availability
 - interpretability
 - regulatory compliance

**The Lifecycle
as an Iterative
Process**

- building an ML system begins with problem framing
 - defining the decision or prediction task
 - establishing meaningful measures of success
 - identifying constraints before selecting a model
- data must then be:
 - collected
 - cleaned
 - transformed
 - converted into useful features
- in production, these transformations should become repeatable pipelines so that new data is processed consistently rather than prepared manually each time
- model development
 - remains important
 - but it is only one stage of the larger workflow.
- once a model is considered suitable it must be:
 - deployed
 - integrated into a service or application
 - scaled appropriately
 - continuously monitored and maintained
- production systems also require a plan for changing conditions
 - new user behavior
 - shifts in the environment
 - declining model performance
 - harmful predictions
 - which may require:

	▪ retraining ▪ new data ▪ rollback procedures ▪ possibly reframing the original problem • for this reason, the ML lifecycle is best understood as: ○ a continuous feedback loop ○ NOT a linear sequence with a final endpoint
Machine Learning Systems	
Testing ML Systems	• quality assurance for machine learning is broader than conventional software testing • in addition to functional correctness, ML systems must be evaluated for: ○ system performance: ▪ latency ▪ throughput ▪ resource usage ▪ availability ○ model performance: ▪ accuracy ▪ false-positive rates ▪ other task-specific metrics ○ data quality: ▪ malformed ▪ missing

	▪ unexpected ▪ or changing inputs ○ bias and edge cases: ▪ behavior that may not appear in ordinary functional tests • because both data and models can evolve after deployment, production testing may also involve techniques such as data validation and shadow deployments for evaluating new models before fully replacing the existing one
Functional Components of an ML System	• a production system can be decomposed into several connected layers: ○ data ingestion ▪ brings raw information into the system ▪ depending on the application, ingestion may be: • streaming • batch-based • triggered on demand ▪ its responsibility is to ensure that the system continuously receives the data needed for downstream processing ○ feature engineering and storage ▪ convert raw data into model-ready representations production systems ▪ may precompute and store frequently used features,

- sometimes through a dedicated feature store so that identical feature definitions can be used during both training and inference

- **training pipelines**

- turn model development into an automated, repeatable process
- they can include:
 - dataset splitting
 - training
 - hyperparameter tuning
 - validation
 - eventually, continuous retraining as new data becomes available

- **model serving**

- exposes the trained model to other applications
 - commonly through an API
- serving infrastructure must be selected according to operational requirements such as:
 - latency
 - scale
 - availability

- **Monitoring and feedback loops**

- connect production behavior back to the development process
- monitoring

	• tracks both infrastructure health and changes in model or data behavior ▪ feedback • supplies eventual outcomes or user responses that can support evaluation and retraining
Data Engineering as the Foundation	• a model is only as dependable as the data pipeline supporting it • production data engineering therefore includes ○ validation rules ○ missing-data handling ○ anomaly detection ○ workflow scheduling ○ mechanisms for enforcing the quality requirements expected by ML models • the model must also operate at realistic scale ○ enterprise systems may process millions of events or hundreds of millions of records requiring ▪ distributed processing frameworks and cloud infrastructure ▪ not achievable with single-machine workflows
Data Lineage and Governance	• an additional production concern is understanding where data came from and how it is allowed to be used • data lineage ○ records the origin and transformation history of data, making it easier to trace downstream problems back to their source

	• data governance ○ defines and enforces: ▪ rules around access ▪ privacy ▪ compliance ▪ anonymization ▪ appropriate use • these controls become particularly important when ML systems use sensitive or regulated information
Data-Specific Failure Modes	• several problems can undermine model performance even when the modeling code itself is unchanged • data poisoning ○ occurs when harmful or misleading observations enter the training or inference pipeline, potentially intentionally ○ systems that learn from open or user-generated data are particularly exposed to this class of risk • poor label quality ○ supervised models trained on incorrect or inconsistent labels are effectively learning from noise ○ label errors ▪ reduce attainable performance ▪ introduce unwanted bias • these issues reinforce the need to treat the training dataset itself as an engineered and monitored component of the system

Cloud Infrastructure	• benefits of cloud infrastructure: ○ elastic compute for scaling resources on demand ○ managed ML services that reduce infrastructure overhead ○ global reach through geographically distributed infrastructure ○ unified data ecosystems that connect ▪ storage ▪ warehouses ▪ lakehouse architectures ▪ analytics ▪ ML tools
Key Takeaways	• production ML can be understood as a set of interdependent engineering components • reliability depends on how well data ingestion, feature computation, training, serving, monitoring, governance, and infrastructure work together—not on the model in isolation
Requirements for Production ML	
Static Data vs. Dynamic Data	• in research ○ models are commonly developed on a fixed dataset with known boundaries ○ after the train/test split is created, the data environment is relatively stable • production systems operate differently ○ new observations continue to arrive ○ user behavior changes ○ seasonality appears

	○ the underlying data distribution may drift ▪ as a result, production ML requires plans for: • periodic or continuous retraining • robust data ingestion pipelines • validation and cleaning of incoming data • handling missing or unavailable data sources • graceful degradation or fallback behavior when dependencies fail ○ the key distinction ▪ research • often treats data as a snapshot ▪ production • treats data as an ongoing stream that can change over time
Model Metrics Are Not Enough	• research workflows often judge models primarily using predictive metrics such as: ○ accuracy ○ F1 score ○ RMSE ○ other task-specific evaluation measures • production systems ○ still care about these metrics ○ they must balance them against operational requirements • important production criteria include: ○ latency

	▪ how quickly predictions are returned ○ throughput ▪ how many requests the system can process ○ scalability ▪ whether the system can handle increasing demand ○ resource usage and cost ○ uptime and reliability • a model with excellent offline accuracy may still be unsuitable if it is ○ too slow ○ too expensive ○ or unreliable in deployment
Continuous Testing and Quality Assurance	• research testing is often centered on evaluation against a holdout dataset and a small number of sanity checks • Production ML requires a broader and continuous testing process ○ pre-production testing ○ canary or shadow deployments ○ continuous monitoring and alerting ○ security testing ○ ongoing checks of data and model behavior • testing therefore continues after deployment rather than ending once a model passes offline evaluation
Key Takeaways	• a production ML system must balance predictive quality with: • changing data

	• operational constraints • continuous validation • the important shift is from optimizing a model once on static data to operating a system that remains: ○ accurate ○ responsive ○ reliable ○ testable as conditions change
ML Prototype, ML Product and ML System	
	•
	•
	•